

Section A - 연결 리스트

[이 섹션에서 지정된 문제 1 개에 답하십시오.]

안내: 문제별 프로그램 템플릿은 APAS 시스템에서 제공됩니다. 반드시 해당 템플릿을 사용하여 함수를 구현하십시오.

1. (insertSortedLL)

사용자로부터 정수 하나를 입력받아 연결 리스트에 오름차순으로 삽입하는 C 함수 insertSortedLL()을 작성하십시오. 현재 연결 리스트에 같은 정수가 이미 존재한다면 삽입해서는 안 됩니다. 함수는 새 항목이 삽입된 인덱스 위치를 반환해야 하며, 정상적으로 수행하지 못한 경우 -1 을 반환해야 합니다. 연결 리스트는 이미 정렬된 연결 리스트이거나 빈 리스트라고 가정할 수 있습니다.

함수 원형은 다음과 같습니다.

```
int insertSortedLL(LinkedList *ll, int item);
```

현재 연결 리스트가 다음과 같다고 가정합니다: 2, 3, 5, 7, 9.

insertSortedLL()을 값 8 로 호출하면 연결 리스트는 다음과 같이 됩니다:

2, 3, 5, 7, 8, 9

함수는 새 항목이 삽입된 인덱스 위치를 반환해야 합니다:

값 8 은 인덱스 4 에 추가되었습니다.

현재 연결 리스트가 다음과 같다고 가정합니다: 5, 7, 9, 11, 15.

insertSortedLL()을 값 7 로 호출하면 연결 리스트는 다음과 같이 그대로 유지됩니다:

5, 7, 9, 11, 15

값 7 은 이미 존재하므로 함수는 정상적으로 삽입을 완료하지 못합니다. 따라서 -1 을 반환해야 합니다:

값 7 은 인덱스 -1 에 추가되었습니다.

예시 입력 및 출력은 다음과 같습니다.

- 1: 정렬된 연결 리스트에 정수 삽입
- 2: 가장 최근에 입력한 값의 인덱스 출력
- 3: 정렬된 연결 리스트 출력
- 0: 종료

선택을 입력하십시오(1/2/3/0): 1

연결 리스트에 추가할 정수를 입력하십시오: 2

결과 연결 리스트: 2

선택을 입력하십시오(1/2/3/0): 1

연결 리스트에 추가할 정수를 입력하십시오: 3

결과 연결 리스트: 2 3

선택을 입력하십시오(1/2/3/0): 1

연결 리스트에 추가할 정수를 입력하십시오: 5

결과 연결 리스트: 2 3 5

선택을 입력하십시오(1/2/3/0): 1

연결 리스트에 추가할 정수를 입력하십시오: 7

결과 연결 리스트: 2 3 5 7

선택을 입력하십시오(1/2/3/0): 1

연결 리스트에 추가할 정수를 입력하십시오: 9

결과 연결 리스트: 2 3 5 7 9

선택을 입력하십시오(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 8
결과 연결 리스트: 2 3 5 7 8 9

선택을 입력하십시오(1/2/3/0): 2
값 8은 인덱스 4에 추가되었습니다.

선택을 입력하십시오(1/2/3/0): 3
결과 정렬 연결 리스트: 2 3 5 7 8 9

선택을 입력하십시오(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 5
결과 연결 리스트: 2 3 5 7 8 9

선택을 입력하십시오(1/2/3/0): 2
값 5은(는) 인덱스 -1에 추가되었습니다.

선택을 입력하십시오(1/2/3/0): 3
결과 정렬 연결 리스트: 2 3 5 7 8 9

선택을 입력하십시오(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 11
결과 연결 리스트: 2 3 5 7 8 9 11

선택을 입력하십시오(1/2/3/0): 2
값 11은 인덱스 6에 추가되었습니다.

선택을 입력하십시오(1/2/3/0): 3
결과 정렬 연결 리스트: 2 3 5 7 8 9 11

2. (alternateMergeLL)

두 번째 연결 리스트의 노드들을 첫 번째 연결 리스트의 한 칸 건너 위치마다 번갈아 삽입하는 C 함수 alternateMergeLL()을 작성하십시오. 두 번째 리스트의 노드는 첫 번째 리스트에서 번갈아 삽입할 수 있는 위치가 있을 때만 삽입해야 합니다.

함수 원형은 다음과 같습니다.

```
void alternateMergeLL(LinkedList *ll1, LinkedList *ll2);
```

예를 들어 두 연결 리스트가 다음과 같다고 가정합니다:

```
LinkedList1: 1, 2, 3  
LinkedList2: 4, 5, 6, 7
```

결과 연결 리스트는 다음과 같습니다:

```
LinkedList1: 1, 4, 2, 5, 3, 6  
LinkedList2: 7
```

첫 번째 리스트가 두 번째 리스트보다 더 길다면 두 번째 리스트는 비어야 합니다. 예를 들어 다음과 같습니다:

```
LinkedList1: 1, 5, 7, 3, 9, 11  
LinkedList2: 6, 10, 2, 4
```

결과:

```
LinkedList1: 1, 6, 5, 10, 7, 2, 3, 4, 9, 11  
LinkedList2: 비어 있음
```

예시 입력 및 출력 세션(초기 리스트가 Linked list 1: 1, 2, 3 / Linked list 2: 4, 5, 6, 7 인 경우):

```
Linked list 1: 1 2 3  
Linked list 2: 4 5 6 7  
선택을 입력하십시오(1/2/3/0): 3  
주어진 연결 리스트를 병합한 결과:  
Linked list 1: 1 4 2 5 3 6  
Linked list 2: 7
```

3. (moveOddItemsToBackLL)

연결 리스트의 모든 홀수를 리스트의 뒤쪽으로 이동시키는 C 함수 moveOddItemsToBackLL()을 작성하십시오.

함수 원형은 다음과 같습니다.

```
void moveOddItemsToBackLL(LinkedList *ll);
```

예시 입력 및 출력은 다음과 같습니다.

```
연결 리스트가 2, 3, 4, 7, 15, 18 인 경우:  
홀수를 뒤로 이동한 결과: 2 4 18 3 7 15
```

```
연결 리스트가 2, 7, 18, 3, 4, 15 인 경우:  
홀수를 뒤로 이동한 결과: 2 18 4 7 3 15
```

```
현재 연결 리스트가 1, 3, 5 인 경우:  
결과: 1 3 5
```

```
현재 연결 리스트가 2, 4, 6 인 경우:  
결과: 2 4 6
```

4. (moveEvenItemsToBackLL)

연결 리스트의 모든 짝수를 리스트의 뒤쪽으로 이동시키는 C 함수 moveEvenItemsToBackLL()을 작성하십시오.

함수 원형은 다음과 같습니다.

```
void moveEvenItemsToBackLL(LinkedList *ll);
```

예시 입력 및 출력은 다음과 같습니다.

연결 리스트가 2, 3, 4, 7, 15, 18 인 경우:

짝수를 뒤로 이동한 결과: 3 7 15 2 4 18

연결 리스트가 2, 7, 18, 3, 4, 15 인 경우:

짝수를 뒤로 이동한 결과: 7 3 15 2 18 4

현재 연결 리스트가 1, 3, 5 인 경우:

결과: 1 3 5

현재 연결 리스트가 2, 4, 6 인 경우:

결과: 2 4 6

5. (frontBackSplitLL)

단일 연결 리스트를 앞쪽 절반과 뒤쪽 절반의 두 하위 리스트로 분할하는 C 함수 frontBackSplitLL()을 작성하십시오. 원소 수가 홀수라면 남은 원소 하나는 앞쪽 리스트에 포함되어야 합니다. frontBackSplitLL()은 frontList 와 backList 두 리스트를 출력합니다.

함수 원형은 다음과 같습니다.

```
void frontBackSplitLL(LinkedList *ll, LinkedList *resultFrontList, LinkedList *resultBackList);
```

예를 들어 주어진 연결 리스트가 다음과 같다고 가정합니다:

2, 3, 5, 6, 7

결과 리스트는 다음과 같습니다:

frontList: 2, 3, 5

backList: 6, 7

예시 입력 및 출력 세션은 다음과 같습니다.

1: 연결 리스트에 정수 삽입
2: 연결 리스트 출력
3: 연결 리스트를 frontList 와 backList 두 개로 분할
0: 종료

선택을 입력하십시오(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 2
결과 연결 리스트: 2

선택을 입력하십시오(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 3
결과 연결 리스트: 2 3

선택을 입력하십시오(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 5
결과 연결 리스트: 2 3 5

선택을 입력하십시오(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 6
결과 연결 리스트: 2 3 5 6

선택을 입력하십시오(1/2/3/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 7
결과 연결 리스트: 2 3 5 6 7

선택을 입력하십시오(1/2/3/0): 2
결과 연결 리스트: 2 3 5 6 7

선택을 입력하십시오(1/2/3/0): 3
주어진 연결 리스트를 분할한 결과:
앞쪽 연결 리스트: 2 3 5
뒤쪽 연결 리스트: 6 7

6. (moveMaxToFront)

정수 연결 리스트를 최대 한 번만 순회한 뒤, 저장된 값이 가장 큰 노드를 리스트의 맨 앞으로 이동시키는 C 함수 moveMaxToFront()를 작성하십시오.

함수 원형은 다음과 같습니다.

```
int moveMaxToFront(ListNode **ptrHead);
```

예를 들어 연결 리스트가 (30, 20, 40, 70, 50)이라면 결과는 (70, 30, 20, 40, 50)이 됩니다.

예시 입력 및 출력 세션은 다음과 같습니다.

1: 연결 리스트에 정수 삽입
2: 가장 큰 값이 저장된 노드를 맨 앞으로 이동
0: 종료

선택을 입력하십시오(1/2/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 30
연결 리스트: 30

선택을 입력하십시오(1/2/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 20
연결 리스트: 30 20

선택을 입력하십시오(1/2/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 40
연결 리스트: 30 20 40

선택을 입력하십시오(1/2/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 70
연결 리스트: 30 20 40 70

선택을 입력하십시오(1/2/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 50
연결 리스트: 30 20 40 70 50

선택을 입력하십시오(1/2/0): 2
결과 연결 리스트: 70 30 20 40 50

선택을 입력하십시오(1/2/0): 0

7. (recursiveReverse)

next 포인터와 head 포인터를 변경하여 주어진 연결 리스트를 재귀적으로 역순으로 만드는 C 함수 recursiveReverse()를 작성하십시오.

함수 원형은 다음과 같습니다.

```
void recursiveReverse(ListNode **ptrHead);
```

예를 들어 연결 리스트가 (1, 2, 3, 4, 5)라면 결과는 (5, 4, 3, 2, 1)이 됩니다.

예시 입력 및 출력 세션은 다음과 같습니다.

1: 연결 리스트에 정수 삽입
2: 연결 리스트 역순 변환
0: 종료

선택을 입력하십시오(1/2/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 1
결과 연결 리스트: 1

선택을 입력하십시오(1/2/0): 1
연결 리스트에 추가할 정수를 입력하십시오: 2
결과 연결 리스트: 1 2

선택을 입력하십시오(1/2/0): 1

연결 리스트에 추가할 정수를 입력하십시오: 3

결과 연결 리스트: 1 2 3

선택을 입력하십시오(1/2/0): 1

연결 리스트에 추가할 정수를 입력하십시오: 4

결과 연결 리스트: 1 2 3 4

선택을 입력하십시오(1/2/0): 1

연결 리스트에 추가할 정수를 입력하십시오: 5

결과 연결 리스트: 1 2 3 4 5

선택을 입력하십시오(1/2/0): 2

역순 변환 후 결과 연결 리스트: 5 4 3 2 1

선택을 입력하십시오(1/2/0): 0

※ 원본 PDF 에 포함된 일부 손글씨/주석은 문자 인식 상태가 불명확하여 본 번역본에서는 본문 문제와 예시 입출력을 중심으로 정리했습니다.